

Reputation-based Byzantine Fault-Tolerance for Consortium Blockchain

Kai Lei*, Qichao Zhang, Limei Xu, Zhuyun Qi

[†] Shenzhen Key Lab for Information Centric Networking & Blockchain Technology (ICNLAB)

School of Electronics and Computer Engineering (SECE)

Peking University, Shenzhen 518055, P.R. China

Email: leik@pkusz.edu.cn, {qczhang, xulimei}@pku.edu.cn, qizy@pkusz.edu.cn

Abstract—The Practical Byzantine Fault Tolerance algorithm (PBFT) has been highly applied in consortium blockchain systems, however, this kind of consensus algorithm can hardly identify and remove faulty nodes in time, and also vulnerable to many attacks against the *primary* node of PBFT. The equality of consortium members' discourse rights is inapplicable to some real scenarios where dominating members are likely to have a larger discourse rights in the voting process.

To address these problems, this paper presents Reputation-based Byzantine Fault Tolerance (RBFT) algorithm that incorporates a reputation model to evaluate the operations of each node in the consensus process. The faulty nodes will get lower discourse rights in the voting process if any malicious behavior is detected, with their reputation decreased. Furthermore, this paper presents an innovative reputation-based primary change scheme. The node with higher reputation obtains greater opportunities to be a primary to generate new valid blocks, which reduces the security risk of the primary.

The experimental results demonstrate that RBFT gains better performance and ensures system security and reliability. Compared with PBFT, it increases the average throughput by 15% and reduces delay by 10%, and the faulty node rate of the system can continue to decrease over time.

Index Terms—consortium blockchain, consensus, Byzantine fault-tolerance, reputation model, primary change

I. INTRODUCTION

Blockchain is a decentralized network system that serves a tamper-resistant and traceable digital ledger. It establishes trust among multiple parties without any authority and extends programmable smart contracts into automated business models. In public blockchain systems, such as Bitcoin [1] and Ethereum [2], any node can freely join and leave from blockchain network, inquiry the historical state, and participate in the blockchain consensus process to determine the current valid state. While consortium blockchain composed by multiple entities is more flexible and controllable to perform business functions since the authority management is set according to the rules of alliance. Thus, it has been widely applied by many commercial organizations such as R3 [3].

The proof-based consensus algorithm adopted by many public blockchain platforms is obviously unsuitable for consortium blockchain due to its low transaction throughput, high delay, and excessive resources consumption. The consensus algorithm for consortium blockchain needs to consider efficiency, cost, and Byzantine fault-tolerance (BFT), thus the BFT state machine replication algorithms in the traditional

distributed domain have become an optimal choice. Through mutual broadcast of several rounds, it is ensured that all the non-faulty nodes reach an agreement after each algorithm instance is finished, which greatly reduces the block confirmation time. The absence of mining process avoids extra overhead, hence the system performance can be not limited.

As one of the most classic state machine replication algorithms, Practical Byzantine Fault Tolerance algorithm (PBFT) [4] has been highly applied in consortium blockchain platforms, and a body of improved algorithms based on it have been proposed. However, they still face the following problems: (1) faulty nodes can destruct system's efficiency and security. It is well-known that these BFT algorithms require a minimum of $3f + 1$ nodes to tolerate only f Byzantine faults. If these faulty nodes cannot be detected and removed in time, they may collude to break system security, however, PBFT lacks some mechanisms to detect and remove them; (2) in many real scenarios, the discourse rights of consortium members are different. For example, shareholders have different discourse rights in the general meeting of stockholders. Normally big institutions or dominating members have a larger voice in the voting process since they are more credible compared with other members, so the reputation of consortium members needs to be considered carefully. However, all the participating nodes in PBFT are completely equal, and different discourse rights cannot be reflected by the reputation, which do not meet the requirements of real scenarios; (3) PBFT executes view-change operation and selects a new *primary* node in rotation only when the current primary fails to complete the consensus. This design brings primary delay attacks, *i.e.*, the primary deliberately delays sending new messages in some consensus rounds to degrade system performance. Besides, the selection sequence of the primary is fixed, thus the new primary may be easily predicted and subjected to malicious attacks.

To counter the above problems, we propose *Reputation-based Byzantine Fault Tolerance* (RBFT) algorithm which incorporates a reputation model of consortium blockchain. On the one hand, RBFT can identify the faulty nodes and maintain the high system availability; on the other hand, the reputation value of participating nodes can be used as the criterion for the judgment of the node's discourse right, which incents nodes to follow the protocol. Besides, by leveraging the ideas from

many proof-based consensus algorithms, such as proof-based writing rights and generating new blocks in rotation, we set a fixed block production period to replace the concept of the view in the BFT algorithms. As the function of the coin-age in Proof-of-Stake (PoS) mechanism, a node's reputation determines the probability that this node is selected as the new primary to generate new blocks, thus reducing the security risk against the primary. Since the system can recover or remove the identified faulty nodes, and the discourse rights of correct nodes are improved, the number of voting nodes for changing consensus state will decrease, leading RBFT achieves better performance in high faulty node rate system.

In summary, we make the following contributions:

- We design a reputation model to evaluate the operation of each nodes in the consensus process. Malicious behavior of faulty nodes will be detected and their reputation value will decrease.
- We propose RBFT algorithm that incorporates the reputation model, and design a built-in discourse right incentive mechanism. The influence of faulty nodes will continue to decrease. We eliminate the concept of view and design an innovative reputation-based primary change scheme.
- We build a simple blockchain system to simulate the consensus process, and compare RBFT with PBFT in terms of transaction throughput, delay and faulty node rate. The results show that RBFT can effectively identify faulty nodes, and the performance is improved in the case of high faulty node rate.

The rest of this paper is organized as follows. Section II and III describes background and system model; section IV articulates the design details of RBFT; section V presents evaluation results; and section VI concludes this paper.

II. BACKGROUND AND RELATED WORK

A. Byzantine Consensus

The distributed consensus in blockchain establishes trust between multiple parties, and generally it is considered as *Byzantine consensus*. Although previous work [5] has proved that there is no consensus solution that can completely tolerate one or more crash failures in a fully asynchronous system, *i.e.*, FLP impossibility, Byzantine consensus is still a hot research topic and has been undoubtedly promoted by the rapid development of blockchain recently. As for the consensus algorithm in blockchain, it can be usually divided into proof-based consensus algorithm and vote-based consensus algorithm.

The concept of proof-based consensus algorithm is that a node with sufficient proofs will get the right to append the new block and obtain rewards. Bitcoin uses Proof-of-Work (PoW) mechanism that requires each miner to compete to solve a difficult puzzle, which consumes a large amount of global resources but can only process nearly 7 transactions per second (TPS). PoS uses the stake to determine mining difficulty. The core concept of PoS is that anyone who possesses more coin-age has higher reputation and more opportunities for generating valid blocks. The stake can also be associated

with a proof for voting, Delegate Proof of Stake (DPoS) [7] uses this concept to select a group of delegates and each delegate has its own fixed period to produce blocks. Normally, the proof-based algorithm guarantees the eventual consistency of the distributed ledger in a large-scale network, but it is limited by its low TPS, high delay and cost. Therefore, it's inapplicable to consortium blockchain.

The vote-based consensus algorithm is more suitable for consortium blockchain. Due to the know-your-customer (KYC) strategy in consortium blockchain, participating nodes can quickly reach a consensus through several rounds of mutual voting. The famous project Hyperledger Fabric [6] uses PBFT in its 0.6 version, and R3 Corda employ BFT-SMaRt [8] which is similar to PBFT. Ripple [9] proposed its consensus algorithm that is based on voting of the nodes in Unique Node List (UNL).

B. Practical Byzantine Fault Tolerance

PBFT is a classic BFT state machine replication algorithm and a mainstream voting-based algorithm. It must guarantee the correctness of the following two attributes:

- *Safety*: all non-faulty replicas execute the requests in the same order (*i.e.*, consensus);
- *Liveness*: clients eventually receive replies to their requests.

To ensure liveness, PBFT uses weak synchrony assumption that messages are guaranteed to be delivered after a certain time bound to circumvent FLP impossibility result. As shown in Fig.1, in the normal cases, the primary orders all the requests from clients, and then all correct nodes reach an agreement via running a three-phase agreement protocol include pre-prepare, prepare, and commit. The primary may crash or perform equivocation, *i.e.*, sending inconsistent messages to different subsets of servers, leading the failure of consensus in current round. Thus, the view-change process is triggered to select a new primary. As a result, PBFT can tolerate $\lfloor \frac{N-1}{3} \rfloor$ Byzantine faults.

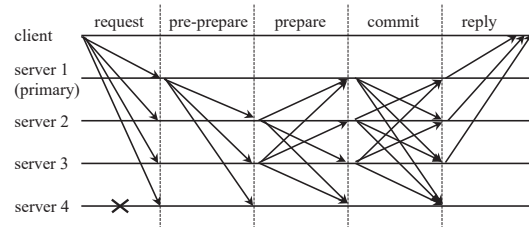


Fig. 1. PBFT normal operation.

There is a large body of work aimed at optimizing PBFT. Some protocols change the paradigm to improve performance and reduce cost [10], while other protocols focus on reducing the minimum of required replicas to $2f + 1$ via building trusted execution environment with security hardware [11]. The primary may maliciously delay the sending of messages without being detected by other nodes. Some protocols are

proposed to solve this problem and achieve load balance by adopting rotating primary and fault detection mechanisms [12]. However, they exert limited influence in identifying faulty nodes, and lack of considering different discourse rights of participating nodes. The design of this kind of algorithm can learn from the ideas of a number of proof-based consensus algorithms in public blockchain, such as stake-based discourse rights and new blocks generated in rotation, but at present, they are hardly considered.

III. SYSTEM MODEL

Fig.2 illustrates our consortium blockchain system model. There are three types of nodes including transaction nodes (*i.e.*, wallet clients $C = \{C_1 \dots\}$), consensus nodes S and storage nodes $U = \{U_1 \dots\}$. The transaction nodes initiate and forward transactions to the consensus nodes. We assume that there are N consensus nodes $S = \{S_1, S_2 \dots S_N\}$ in the blockchain system, where a primary S_p exists in each round. All the consensus nodes will cache the received transactions to their local transaction pool, and the primary S_p orders valid transactions and packs them into a block. If the block is verified by most consensus nodes, it is considered valid and the storage nodes will finally store the block. In the blockchain system, consensus nodes may collude to eclipse other honest nodes due to malicious attacks or personal benets, but the number of faulty consensus nodes is limited that no more than f and must satisfy $N > 3f$. All the messages are eventually delivered within an unknown upper bound T_{delay} . In addition, Byzantine nodes cannot break the standard assumptions that the hash functions are collision-resistant and the signature cannot be forged. We denote a message m signed by consensus node i as $\langle m \rangle_{\sigma_i}$.

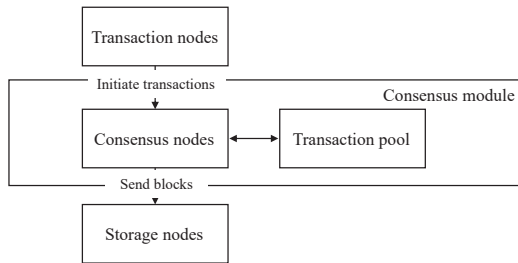


Fig. 2. Simple consortium blockchain system model.

The system uses diverse machines and reliable operating systems to prevent possible common-mode attacks, and it is difficult for external adversaries to hijack multiple nodes at the same time. The Network Time Protocol (NTP) server is used to synchronize the time of consensus nodes. We guarantee that this service is reliable, when a consensus node is restart, it will firstly adjust its time to synchronize with the NTP server, then participate in consensus process. In addition, each consensus node maintains a local Consensus Node List (CNL) which contains the number of current consortium consensus nodes, and the identity number, IP address/port, public key, reputation

value and trusted state of each consensus node. The CNL represents the current participating consensus nodes' state, and it is significant to guarantee the consistency of CNL, so we set the initial configurations of all the nodes' CNL to be the same.

IV. THE BASIC OF RBFT

A. Consensus Process

RBFT removes the concept of view in PBFT and sets a fix block production period to generate new block in rotation, which is similar to DPoS. In every round, S_p generates a new block and broadcasts to all consensus nodes. All the consensus initialize their local Digest Vectors (DVs) at the beginning of each consensus round. We assume that $DV_i[j]$ is used for consensus node i to store the digest of block header in the pre-prepare (if j is S_p) or prepare message from consensus node j . After each round of consensus, all consensus nodes will calculate and update nodes' reputation value according to the reputation model and received DVs, and the reputation value ultimately determines the discourse rights. As shown in Fig.3, there are three major states in the consensus process: pre-prepare, prepare and commit. The condition for a consensus node i to update consensus state is that i must receive the matching protocol messages from enough consensus nodes, and their total discourse rights (include i) are not less than a certain threshold $D_{threshold}$. $D_{threshold}$ can be specified as follows:

$$D_{threshold} = \frac{1}{N} * (2 \lfloor \frac{N-1}{3} \rfloor + 1)$$

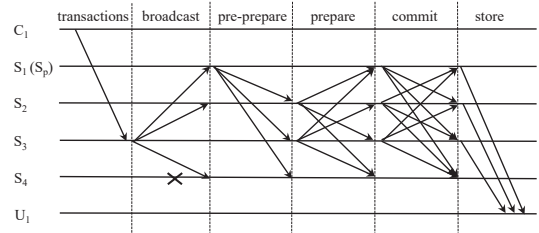


Fig. 3. RBFT consensus process.

The above consensus state update condition is different from that of PBFT which receives $2 \lfloor \frac{N-1}{3} \rfloor + 1$ matching messages from different consensus nodes. The more specific consensus process of RBFT contains the following steps:

S1. The transaction node C_1 initiates a transaction tx and broadcasts to the blockchain network;

S2. The system selects the primary S_p of the current round of consensus.

S3. After receiving the transaction tx , a consensus node verifies whether the transaction is valid. If the transaction is valid, it is written into the local transaction pool and forwarded to other consensus nodes, if not, it will be directly discarded. The primary S_p packs local valid transactions into a block, and send the header of this block (*i.e.*, B_{header}) to the NTP server to fetch time information.

S4. NTP server provides the current timestamp, and generates a signature of B_{header} and the timestamp. A new block header B_{new_header} is constructed with B_{header} , timestamp and the signature. Then the NTP server returns the B_{new_header} to S_p .

S5. After receiving the B_{new_header} , S_p produces a new block (i.e., B). Then it generates and broadcasts a pre-prepare message with B piggybacked to all the consensus nodes. The message has the form $\langle\langle PRE-PREPARE, h, d, p \rangle_{\sigma_p}, B\rangle$, where h is the height of the new block, d is the digest of the B_{new_header} , and p is the sequence number of the current primary.

S6. After the node i receives the pre-prepare message, it first verifies the identity of the primary, and then checks the validity of the block. If the block is valid, it will send a prepare message $\langle PREPARE, h, d, i \rangle_{\sigma_i}$ to other consensus nodes, and update its local $DV_i[p] = d$ at the same time;

S7. The consensus node i collects the prepare messages from other nodes, verifies these messages and updates its local DV. The received matching messages correspond to the nodes with different discourse rights. The node i calculates the sum of the discourse rights (including itself), and if the sum is not less than $D_{threshold}$, it will update consensus state to commit and send a commit message with its local DV piggybacked to all the consensus nodes. The message form is $\langle\langle COMMIT, h, d, i \rangle_{\sigma_i}, \langle DV \rangle_{\sigma_i}\rangle$.

S8. Similarly, when a consensus node receives match messages with $D_{threshold}$ discourse rights, this node will update consensus state to local-commit and enter the store phase. The new block B is append to the blockchain and sent to the storage nodes for storage.

S9. After reaching to local-commit state, all the consensus nodes will compare their received DV, calculate the reputation values and discourse rights of each consensus node according to the reputation model, and update their local CNL. The failure of any of the above steps may lead to the failure of the consensus in current round. Eventually there is no valid block in this round, and the new block will be postponed until the next round.

The view-change process of PBFT is equivalent to a round of consensus, and requires mutual broadcast of extra sets of protocol messages to ensure that requests that commit are totally ordered across views, which is very time consuming. By using the trusted NTP server to synchronize the clock, RBFT can configure different block production periods to avoid the delay caused by view-change process. Both the blocks and DVs are transmitted by piggybacking on protocol messages to reduce communication overhead. Since the values in the DVs are substantially equal in the case of no faulty node, encoding techniques can be used to optimize and reduce the size of messages transmitted during the commit phase.

B. The Verification of Transaction and Block

Transaction verification. Consensus nodes need to verify all the received transactions, and a valid transaction must satisfy four conditions: (1) right signature; (2) correct transaction

format with valid timestamp; (3) the scripts in the transaction can be executed correctly; (4) the transaction never appear in historical blocks.

Block Verification. When the consensus node receives the packed block from the primary S_p in pre-prepare phase, it will verify whether the $hashPrevBlock$ in the B_{new_header} points to the latest block in the highest block height. If not, it will cache this block and synchronize historical blocks to ensure that it reaches the latest state of blockchain network. Besides, it needs to verify the NTP signature of B_{new_header} , and the timestamp must be within the current block production period. As a valid block, another two conditions must be reached: (1) all transactions inside is valid; (2) the accumulated digest of all the transactions is equal to the $hashMerkleRoot$ in the B_{new_header} .

C. Reputation Model

The reputation value is one way to express the credibility of node and used to confirm the discourse right of consensus node. In RBFT, the reputation value R is a real number between 0 and 1, the larger the number, the higher the credibility. As for the newly added consensus node of system, its reputation value is initialized as 0.5. Due to the different behavior of the primary and other nodes during the consensus process, we discuss the two situations respectively. Let $R_i(t)$ be the reputation of a consensus node S_i in the consortium blockchain after the t th round of voting, then $R_i(t+1)$ can be specified as follows:

- if S_i is the primary, then
$$R_i(t+1) = \begin{cases} \min(1, (1+y)R_i(t)), & \text{if its proposed block is appended to blockchain;} \\ xR_i(t), & \text{if it doesn't generate a new block;} \\ 0, & \text{if the node sends inconsistent messages to different nodes.} \end{cases}$$
- if not, then
$$R_i(t+1) = \begin{cases} \min(1, (1+y)R_i(t)), & \text{if the node sends consistent messages and agrees with the majority;} \\ xR_i(t), & \text{if the node doesn't vote or it disagrees with the majority;} \\ 0, & \text{if the node sends inconsistent messages to different nodes.} \end{cases}$$

- Where $0 < x < 1$ and $0 < y < 0.03$.

If the primary does not generate a new block in this round, or other consensus nodes do not vote or the votes are inconsistent with the final result, the reputation value of the corresponding nodes will decrease. The value of x determines the speed at which the reputation value is reduced in this case, and can be set according to the application requirements. When an inconsistent vote is detected from the same consensus node, this node will be considered Byzantium and its reputation will be directly reduced to zero. When the primary generates a valid block, or the voting of other nodes is the same as the final result, the reputation of these nodes will be gradually increased, and the speed of the increase depends on the value of y .

The trusted state of the consensus node is determined by the reputation value R . We distinguish the following four trusted states:

- **Good:** $R \in [\alpha, 1]$, its trusted state value TS is 1;
- **Normal:** $R \in [0.5, \alpha)$, $TS = 2$;
- **Abnormal:** $R \in [\beta, 0.5)$, $TS = 3$;
- **Error:** $R < \beta$, $TS = 4$.

Where $0.5 < \alpha < 1$ and $0 < \beta < 0.5$.

Fig.4 illustrates the transformation of nodes' trusted state. When a new consensus node is added to the system, its default state is normal. If the node keeps good behavior, its reputation value will exceed the threshold α and can be transformed to a good state, and the node with a good state has more opportunities to be selected as a primary. If the node performs abnormal behaviors, it will be converted to abnormal state since its reputation value continuously decreases. When the reputation value is lower than the threshold β , the node will be judged as a faulty node. Finally, all the consensus nodes update the nodes reputation and state information in their local CNL after each round of consensus.

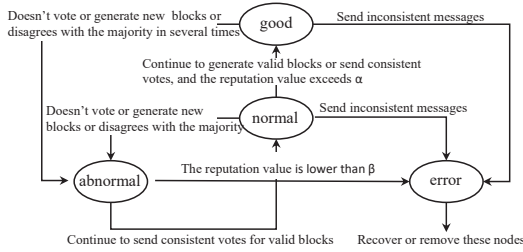


Fig. 4. The transformation of nodes' trusted state.

D. Identifying Faulty Consensus Nodes

RBFT mainly has three phases: pre-prepare, prepare and commit. The Byzantine nodes can send inconsistent message in any one of them. In the above we describe the design of the reputation model, but how to detect the different malicious behavior of the primary or other consensus nodes becomes a critical problem to be solved. Since RBFT uses extra data structure (DVs) to collect and record the digest information of B_{new_header} from the voting blocks of each node, it can easily detect the malicious behaviors of the Byzantine nodes by comparing the DVs of all the nodes. Even if all the nodes are fail to enter the commit phase and reach in a consensus in a consensus round, all the nodes are still required to broadcast the $\langle DV \rangle_{\sigma_i}$ message at the end of this round to ensure that the node can get the DV information of all the correct nodes.

The primary is responsible for generating new blocks, and its errors can be classified into the following two categories:

- The primary does not send pre-prepare and packed block messages due to crash or intentional misconduct, which results in that there is no new block generated in this consensus round. We can detect $DV[p]=0$ to discover this abnormal behavior and decrease the reputation of the primary.
- The primary sends different blocks to different subset of consensus nodes in the pre-prepare phase. In this case, other nodes will detect the difference in $DV[p]$ received

from different nodes, so this behavior can also be detected and penalized, and the reputation value will be directly reduced to zero.

Similarly, other nodes also have two kinds of faulty behaviors: (1) the node does not send a vote, this behavior will be detected by verifying whether the $DV[i]$ of the node is 0; (2) the node sends an inconsistent vote, *i.e.*, equivocates in the prepare or commit phases. The malicious behavior of the node in the prepare phase can also be detected by comparing DV, and the malicious modification of the DV information can also be detected by requesting evidence compulsively. The specific process for identifying faulty consensus nodes is as follows:

S1. After receiving the DV information of other consensus nodes, the node k compares them with its local DV.

S2. If the node found that all the received DV information is the same during the comparison process, it will update the reputation values of all the nodes directly according to the reputation model. The following operations are not required since there is no faulty node in the consensus process in this round.

S3. If the node k found the difference between the received $DV_i[j]$ and the local $DV_k[j]$, it indicates two cases: (1) node j sends different voting information to different consensus nodes in the prepare phase; (2) node i tampers with its DV_i and sends a fake DV information attempting to frame node j . Thus, the node k will ask node i for evidence and sends local received prepare message from node j to node i . If node i is correct, it will also send the received prepare message from node j to node k .

S4. If the node k receives the prepare information from node i , and this message is valid and consistent with the information in $DV_i[j]$, the node k will believe that the node j equivocates in the prepare phase, and node j will be marked as error state.

S5. If the node k does not receives valid evidence from node i within the current round time period, node i will be considered to have sent fake DV information during the commit phase, thus it will be marked as error state.

S6. Finally, the node k will broadcast the faulty node information and corresponding evidence found in this round to other nodes, and all the correct consensus nodes will verify and update the reputation value and trusted state of each node according to the reputation model.

E. The Selection of the Primary

In PBFT, the new primary is selected in the form of $p = v \bmod N$, where v is the new view number. The way that selecting new primary in rotation brings security risks. The core principle of RBFT's primary change scheme is that the higher the node's reputation value, the easier it is to be selected as a primary, and the selection result is unpredictable. For a node i , assume its trusted state value is TS_i , then the probability that it is selected as new primary is as follow:

$$P(p = i) = \frac{\lambda}{TS_i}$$

Therefore, for nodes a and b , if $TS_a < TS_b$, then $\frac{\lambda}{TS_a} > \frac{\lambda}{TS_b}$, *i.e.*, $P(p = a) > P(p = b)$. Suppose $NodeSum_z$ is the number

of consensus nodes with their $TS=z$, then λ can be calculated as follows:

$$\lambda = \frac{1}{\sum_{z=1}^4 \frac{NodeSum_z}{z}}$$

In order to ensure the selection process in an unpredictable order, a random number Rd is generated for determining the primary. Suppose the latest block in the highest block height is B , then $TimeStamp$ and $Signature$ are generated by NTP server and contained in the B_{new_header} . Rd can be specified as follows:

$$Rd_{source} = TimeStamp \oplus Signature$$

$$Rd = StrToInt(SubStringEnd32(Hash_{t+1}(Rd_{source}))) \bmod N$$

Since the value of the B_{new_header} is unpredictable and the NTP server is trusted, the random number Rd becomes unpredictable. If there is no new block generated in the previous round, then the node needs to count the times t that the system does not produce blocks in continuous consensus rounds. In this case, the hash operation in the process of calculating the random number Rd will be executed $t+1$ times. According to the probability that each node is selected as a primary, the sequence number of the primary in current consensus round will be finally determined. Suppose the node i is the new primary, it requires to satisfy the following condition:

$$\sum_{j=1}^{i-1} P(p=j) \leq \frac{Rd}{N} < \sum_{j=1}^{i-1} P(p=j) + P(p=i)$$

F. The Discourse Right and Incentive Mechanism

In RBFT, the discourse right of a node is determined by its reputation value. The higher the reputation value, the higher the discourse right. The calculation and update of the discourse right is completed before the next round of consensus. Suppose $D_i(t+1)$ represents the discourse right of node i in the $t+1$ th consensus round, it can be specified as follows

$$D_i(t+1) = \frac{R_i(t)}{\sum_{i=1}^N R_i(t)}$$

The sum of all the nodes' discourse rights is 1. In the consensus process, PBFT requires that $2\lfloor \frac{N-1}{3} \rfloor + 1$ matching valid protocol messages must be received to change consensus state, while RBFT only requires that the sum of all the received protocol messages' discourse rights exceeds $D_{threshold}$. As the system continues to run, the discourse rights of the system will be more concentrated on correct nodes, leading a reduction in the number of protocol messages required in the prepare and commit phases, and finally the consensus delay is also decreased.

Incentive and punishment mechanisms can better regulate the behavior of alliance nodes. We consider that incentives are reflected by the change of each node's reputation value. The increase of reputation value makes nodes get a good trusted state, a greater voice, and a higher probability of generating new blocks. In addition, we can also consider that alliance members can jointly set up the initial reward fund,

and the alliance member who generates more blocks gets more substantial rewards, which is similar as the mining rewards in public blockchain systems. As for punishment, the node with a reputation value below the threshold will be considered as a faulty node, thus it will obtain fewer rewards and will be recovered or removed by the system.

G. Correctness

Lemma 1: The proportion of all the correct nodes discourse right in the consensus process does not decrease over time.

Proof: Since the correct nodes will always operate in the direction specified by the protocol, the reputation value of the correct nodes will rise with time and reach a good trusted state, and their discourse rights will also rise accordingly. Since valid evidences of a Byzantine nodes malicious behaviors must be publicized before it is confirmed as a faulty node, the correct nodes cannot be affected and framed successfully by Byzantine nodes. As long as the faulty nodes deviate from the direction of the protocol, they will be detected and their reputation value will be lowered, resulting lower discourse rights, as a result, the system can effectively identify these faulty nodes and take timely measures. In this case, the proportion of all the correct nodes' discourse right will rise with time. Even if the faulty nodes "pretend" correct nodes in order to get higher discourse right, their reputation value can only rise as fast as the correct nodes, thus they cannot increase the proportion of their discourse rights to defeat the correct nodes.

Lemma 2: If the number of faulty nodes does not exceed $\lfloor \frac{N-1}{3} \rfloor$, RBFT can guarantee the security and liveness of the consortium blockchain system.

Proof: At the time of system initialization, all the nodes have the same reputation value and discourse rights. Therefore, $2\lfloor \frac{N-1}{3} \rfloor + 1$ correct nodes can ensure that more than $D_{threshold}$ of the discourse rights in the system are concentrated on the correct nodes. In [4], the author proved that PBFT can guarantee the security and liveness of the system in this case. Since RBFT does not relax the conditions of PBFT, it can also guarantee security and liveness. In another case, due to the establishment of the reputation model in RBFT, the proportion of all the correct nodes' discourse rights in the consensus process will not decrease over time (*i.e.*, Lemma 1), so the algorithm can also continue to ensure the security and liveness of the system.

V. PERFORMANCE EVALUATION

We implemented a simple consortium blockchain prototype system, which mainly includes a transaction module and a consensus module. The transaction module is responsible for generating transactions and sending them to the consensus module for verification and consensus. The consensus module encapsulates different inner consensus algorithms (including RBFT and PBFT) to verify the transaction, generate and commit new blocks. We created multiple processes running on a four-core machine (Intel Core i5-6500, 3.2 GHz, 8 GB RAM) to simulate different consensus nodes. The transaction module is implemented through a transaction process, where five

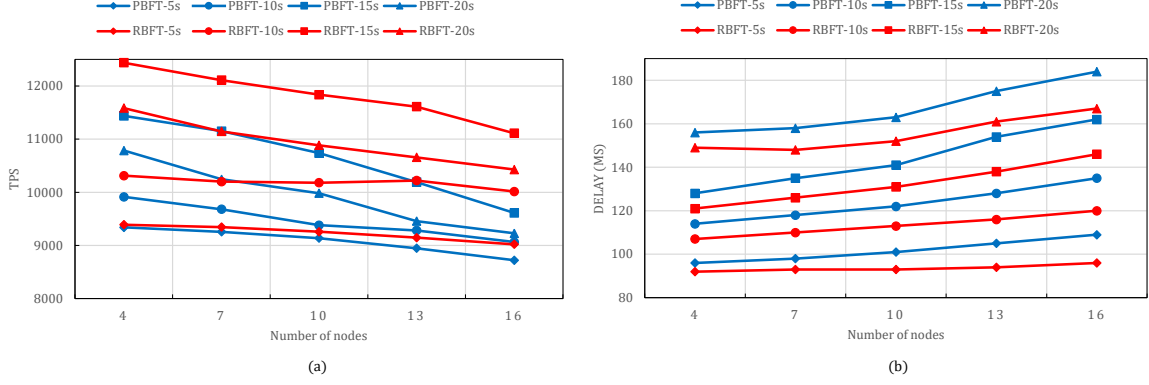


Fig. 5. (a) TPS comparison under different block production period and numbers of nodes; (b) delay comparison under different block production period and numbers of nodes.

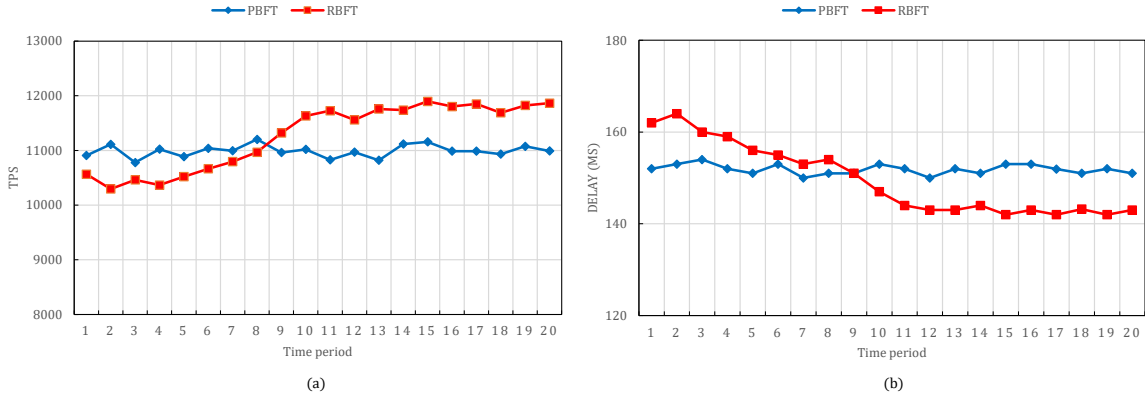


Fig. 6. (a) the change of TPS over time; (b) the change of delay over time.

trade accounts are created. In order to continuously generate transactions, we set these trade accounts have enough initial quota and the transaction amount is 1 in each transaction. The consensus module simulates multiple consensus nodes through multiple consensus execution processes.

We will evaluate the performance of RBFT in three aspects, including transaction throughput, delay and faulty node rate. In blockchain systems, the transaction throughput is usually represented by TPS, and the delay indicates the time required for a new block from being generated to being confirmed. The lower delay indicates the shorter execution time of the consensus algorithm in each consensus round. The block generation delay $Delay_{block}$ contains the consensus algorithms execution time $T_{consensus}$ and the block broadcast time $T_{broadcastBlock}$. When calculating $Delay_{block}$, the block production period is not be considered. Suppose the time at which a block is confirmed is $T_{blockConfirm}$, and the time at which the primary packs a new block is $T_{blockPack}$, then $Delay_{block}$ can be specified as follows:

$$Delay_{block} = T_{blockConfirm} - T_{blockPack} = T_{consensus} + T_{broadcastBlock}$$

The faulty node rate is an important indicator to measure

the security and reliability of the system. It is the ratio of the number of faulty nodes to the total number of consensus nodes in the current blockchain system. The lower the faulty node rate, the safer and more reliable the blockchain system.

A. Impact of Block Production Period and Number of Nodes

We evaluate the TPS and delay of our blockchain system with different block production period and number of nodes. The number of nodes is 4, 7, 10, 13 and 16, correspondingly, its number of error nodes is 1, 2, 3, 4 and 5. The block production period is incremented from 5 to 20 seconds in every 5 seconds. When the system runs stably, we count 20 blocks and calculate the average throughput TPS and delay. Fig.5 illustrates the experimental results of RBFT and PBFT under different block production period and number of nodes.

Under the same conditions, different block production periods are set. The TPS of RBFT is higher than that of PBFT, and the delay is always lower than that of PBFT. As the number of nodes increases, the advantages of RBFT are even more significant. Therefore, the overall performance of RBFT has an obvious advantage over the performance of PBFT in high faulty node rate blockchain systems. When

the block production period is 15 seconds, RBFT's TPS can reach more than 10000, which is the maximum value. The block generation delay is between 120ms and 150ms. When the block production period is controlled between 5s and 20s, the systems TPS can be guaranteed to be above 8000, and the delay is within 220ms. Compared with PBFT, it achieves a 15% higher throughput and 10% lower delay. Hence, RBFT can meet the performance requirements of most applications and guarantee system security and continuous reliability.

B. Changes of TPS, Delay and Faulty Node Rate over Time

We also evaluate the changes of the system's TPS, delay and faulty node rate over time in the configuration that the block production period is 15s, the number of nodes is 16, and the number of faulty nodes is 5. Fig.6(a) illustrates the change of TPS over time. The TPS of PBFT is stable at about 11000, while the TPS of RBFT is initially 10000, but the overall trend is increasing. After the 11th time period, RBFT's TPS gets into stable at around 11800, slightly higher than that of PBFT.

Fig.6(b) illustrates the change of the delay of the two algorithms over time. The average delay of PBFT is stable at around 152ms, while RBFT initially has a higher delay about 163ms. As time goes by, the average delay of RBFT is gradually decreasing to 140ms and becoming stable after the 11th time period.

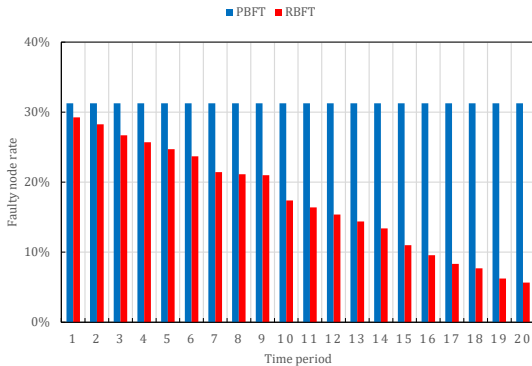


Fig. 7. The change of faulty node rate over time.

Fig.7 illustrates the change in the faulty node rate. The results show that the faulty node rate of PBFT does not change with the running time of the system, while RBFT can effectively identify and remove the faulty nodes in the system, so that the average faulty node rate in the system is gradually reduced, and the security and availability of the system can be ensured.

Since RBFT requires to calculate and transmit DVs, which brings extra overhead, its performance is slightly worse than that of PBFT at the beginning. With the system continuously runs, RBFT that integrates the reputation model can effectively identify the faulty nodes, thus the system can actively recover or eliminate these nodes to reduce the faulty node rate. The faulty nodes will be punished to decrease their reputation, so that they have little probability to be a primary to perform

delay attacks. More important, they have fewer opportunities to impede the generation of new blocks in each time period. Because the discourse right of correct nodes increases, the condition of consensus state update is easier and quicker to be satisfied. Finally, the performance of RBFT gradually improves and exceeds PBFT. Compared with PBFT, RBFT achieves better system performance, security and reliability.

VI. CONCLUSION

In this paper, we propose a new consensus algorithm RBFT for consortium blockchain. We design a reputation model to score the behavior of all the consensus nodes in the consensus process, and the faulty nodes will get lower reputation if any malicious behavior is detected. By learning from many public blockchain consensus algorithms, we consider the distinction of alliance members' discourse right and the necessary incentive mechanism, and an innovative reputation-based primary change scheme is proposed. Since the selection of the primary is unpredictable, the security risks of the primary are reduced. To satisfy the requirement of many real scenarios, we establish the principles that the alliance members with higher reputation can gain greater discourse rights during the voting process and are more likely to become the primary to generate new blocks. Our experiments demonstrate that RBFT ensures the security and reliability of the consortium blockchain system since the faulty nodes can be effectively identified and handled in time. In high faulty node rate systems, RBFT achieves higher TPS and lower delay than PBFT as the system continues to operate. The future work will focus on the deployment and optimization of RBFT in real consortium blockchain systems.

ACKNOWLEDGMENT

This work has been financially supported by Natural Science Funding of Guangdong Gov (No: 2018A030313017) and Shenzhen Key Fundamental Research Projects (No. JCYJ20170412151008290, JCYJ20160330095313861).

REFERENCES

- [1] Bitcoin. <https://bitcoin.org/en/>.
- [2] Ethereum. <https://www.ethereum.org/>.
- [3] R3. <https://www.r3.com/>.
- [4] M. Castro, B. Liskov, and M. Pease, "Practical Byzantine fault tolerance and proactive recovery," *ACM Trans. Comput. Syst.*, vol. 20, pp. 398-461, 2002.
- [5] M. J. Fischer, N. A. Lynch, and M. S. Paterson, "Impossibility of distributed consensus with one faulty process," *J. of the ACM*, vol. 32, pp. 374-382, 1985.
- [6] Hyperledger Fabric. <https://www.hyperledger.org/projects/fabric/>.
- [7] D. Larime, "Delegated Proof-of-Stake (DPOS)," Bitshare whitepaper, 2014.
- [8] A. Bessani, J. Sousa, and E. E. P. Alchieri, "State machine replication for the masses with BFT-SMART," in *Proceedings of 2014 44th Annual IEEE/IFIP International Conference on Dependable Systems and Networks*, pp. 355-362, 2014.
- [9] Ripple. <https://ripple.com/>.
- [10] T. Distler, C. Cachin, and R. Kapitza, "Resource-efficient Byzantine fault tolerance," *IEEE Trans. Comput.*, vol. 65, pp. 2807-2819, 2016.
- [11] J. Behl, T. Distler, and R. Kapitza, "Hybrids on steroids: Sgx-based high performance bft," *Proc. 12th EuroSys*, 2017.
- [12] G. S. Veronese, M. Correia, A. N. Bessani, and L. C. Lung, "EBAWA: Efficient Byzantine agreement for wide-area networks," in *Proc. 12th Symp. on High Assurance Systems Engineering (HASE10)*, pages 10-19, 2010.